

中国科学院大学

《计算机组成原理(研讨课)》实验报告

姓名 钱宸玮 学号 2024K8009929010 专业 计算机科学与技术
实验项目编号 3 实验名称 定制功能型处理器设计

- 注 1: 撰写此 Word 格式实验报告后以 PDF 格式保存 SERVE CloudIDE 的 /home/serve-ide/cod-lab/reports 目录下 (注意: reports 全部小写)。文件命名规则: prjN.pdf, 其中 prj 和后缀名 pdf 为小写, N 为 1 至 4 的阿拉伯数字。例如: prj1.pdf。PDF 文件大小应控制在 5MB 以内。此外, 实验项目 5 包含多个选做内容, 每个选做实验应提交各自的实验报告文件, 文件命名规则: prj5-projectname.pdf, 其中“-”为英文标点符号的短横线。文件命名举例: prj5-dma.pdf。具体要求详见实验项目 5 讲义。
- 注 2: 使用 git add 及 git commit 命令将实验报告 PDF 文件添加到本地仓库 master 分支, 并通过 git push 推送到实验课 SERVE GitLab 远程仓库 master 分支 (具体命令详见实验报告)。
- 注 3: 实验报告模板下列条目仅供参考, 可包含但不限定如下内容。实验报告中无需重复描述讲义中的实验流程。

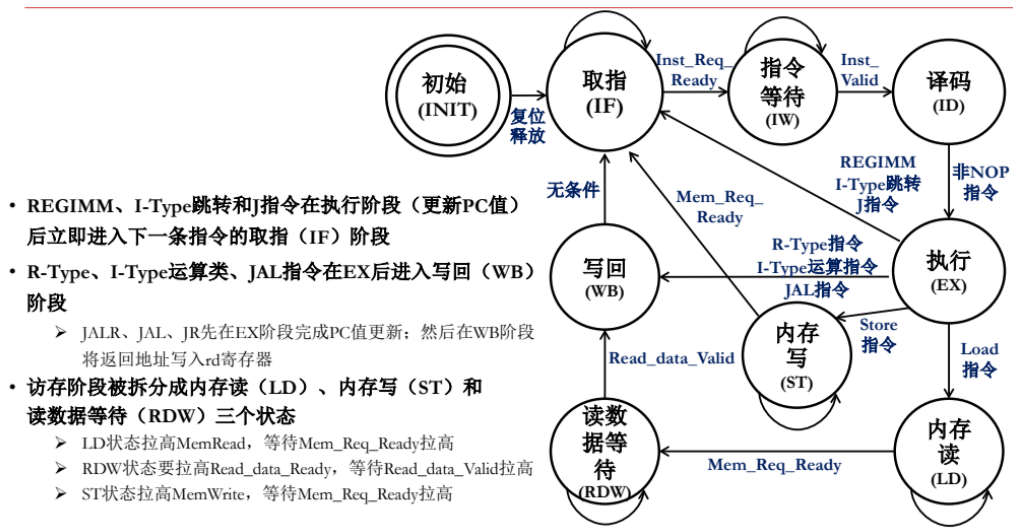
完整实验流程包括设计、开发、调试三个步骤:

- 设计步骤为在代码编写前的准备阶段, 主要包括模块划分和模块结构设计, 并绘制设计图纸。
- 开发步骤为在 S-IDE 集成开发环境中进行 Verilog/C 代码编写, 并实现完整功能。
- 调试步骤包括本地仿真验证后推送至云平台进行 FPGA 原型验证, 以及后续迭代调试的全过程。

一、设计成果(对设计流程的总结。需附上设计结构图, 可用 PPT 或者在线绘图软件绘制逻辑结构框图后保存为 PDF, 再插入到 L^AT_EX 中)

1 MIPS/RISC-V 定制功能型处理器设计

访问真实内存的状态机状态转移图如下:



与实验二的多周期处理器相比, 新增了 IW, RDW 两个等待握手状态; 原 MEM 访存状态被替换为了 ST 和 LD 两个状态。

访问真实内存需要引入若干握手信号,其中由 CPU 发出的有 Inst_Req_Valid(CPU 确认取指),Inst_Ready (CPU 已取到指令),Read_data_Ready(CPU 准备从内存读取数据);由内存发出的有 Inst_Req_Ready(内存已准备好发出指令),Inst_Valid(内存已发送指令),Mem_Req_Ready(内存已准备好发出数据),Read_data_Valid (内存已发出数据)。

实验中,实现了两种性能计数器,周期计数器和指令计数器。其中,周期计数器在每个时钟上升沿自增即可,指令计数器则需要在状态机处于 IF 状态时根据 Inst_Req_Valid 确定是否自增。

2 软件部分设计

1.perf_cnt.h

定义了测试例运行的相关状态,函数等。状态包括是否通过,程序运行周期数和指令数。

2.perf_cnt.c

包含四个函数,其中 unsigned long __uptime() 通过访问基地址获取 CPU 运行周期数,unsigned long __ins() 访问 PERF_CNT_BASE + 0x0008 获得 CPU 运行周期数,void bench_prepare(Result *res) 在测试例运行前,将当前的时间周期数和指令执行数记录进 Result 结构体中,void bench_done(Result *res) 在测试例运行完毕后调用,读取当前的周期数和指令数,并减去 bench_prepare 中记录的初始值。

3.bench.c

基准测试框架的入口程序,TODO 部分补全了结果的打印。

4.printf.c

驱动程序,TODO 部分补全了 puts 函数,定义指向 UART 发送 FIFO 和状态寄存器的指针,循环遍历字符串中的每个字符,将字符写入发送 FIFO 并返回发送的字符数。

二、 开发成果(对代码开发流程的总结。需附上包含注释的 Verilog HDL 关键代码段、相应信号的仿真波形和信号变化的说明)

1 MIPS/RISC-V 定制功能型处理器设计

1. 修改后的状态机

状态机新增状态如下:

```
IW: begin
    if (Inst_Valid == 1'b1) begin
        next_state = ID;
    end
    else begin
        next_state = IW;
    end
end
```

图 2: IW 状态

```

LD: begin
    if(Mem_Req_Ready == 1'b1) begin
        next_state = RDW;
    end
    else begin
        next_state = LD;
    end
end
end

```

图 3: LD 状态

```

ST: begin
    if(Mem_Req_Ready == 1'b1) begin
        next_state = IF;
    end
    else begin
        next_state = ST;
    end
end
end

```

图 4: ST 状态

```

RDW: begin
    if(Read_data_Valid == 1'b1) begin
        next_state = WB;
    end
    else begin
        next_state = RDW;
    end
end
end

```

图 5: RDW 状态

这些状态都在收到对应的握手信号后更新,否则维持原状态,等待内存响应。

2. 性能计数器

性能计数器如下:

```
// 性能计数器
reg [31:0]      cycle_count; //周期计数
assign          cpu_perf_cnt_0      =      cycle_count;
always @ (posedge clk) begin
    if(rst == 1'b1) begin
        cycle_count <= 32'b0;
    end
    else begin
        cycle_count <= cycle_count + 32'd1;
    end
end
end
```

图 6: 周期计数器

```
reg [31:0]      instruction_count; //指令计数
assign          cpu_perf_cnt_1      =      instruction_count;
always @ (posedge clk) begin
    if(rst == 1'b1) begin
        instruction_count <= 32'b0;
    end
    else begin
        if(current_state == IF && Inst_Req_Ready == 1'b1) begin
            instruction_count <= instruction_count + 32'd1;
        end
    end
end
end
```

图 7: 指令计数器

指令计数器必须在 Inst_Req_Ready 拉高时自增,否则可能会在内存未响应,IF 状态循环时将一条指令计为多条。

2 软件部分设计

1.perf_cnt.h

补全如下:

```
typedef struct Result {
    int pass;
    unsigned long msec;
    unsigned long ins;
} Result;
```

图 8: perf_cnt.h

2.perf_cnt.c

补全如下:

```
unsigned long _uptime() {
    // TODO [COD]
    // You can use this function to access performance counter related with time or cycle.
    volatile unsigned long *cycle_counter = (volatile unsigned long *)PERF_CNT_BASE;
    return *cycle_counter;
    return 0;
}

unsigned long _ins() {
    volatile unsigned long *ins_counter = (volatile unsigned long *) (PERF_CNT_BASE + PERF_CNT_OFFSET_1);
    return *ins_counter;
}

void bench_prepare(Result *res) {
    // TODO [COD]
    // Add preprocess code, record performance counters' initial states.
    // You can communicate between bench_prepare() and bench_done() through
    // static variables or add additional fields in `struct Result`
    res->msec = _uptime();
    res->ins = _ins();
}

void bench_done(Result *res) {
    // TODO [COD]
    // Add postprocess code, record performance counters' current states.
    res->msec = _uptime() - res->msec;
    res->ins = _ins() - res->ins;
}
```

图 9: perf_cnt.c

3.bench.c

补全如下:

```

    if (succ) printk(" Passed.\n");
    else printk(" Failed.\n");

    pass &= succ;

    // TODO [COD]
    // A benchmark is finished here, you can use printk to output some information.
    // `msec' is intended indicate the time (or cycle),
    // you can ignore according to your performance counters semantics.
    printk("TOTAL CYCLES           : %u\n", msec    );
    printk("TOTAL INSTRUCTIONS      : %u\n", ins     );
}
}

```

图 10: bench.c

4.printf.c

补全如下:

```

puts(const char *s)
{
    //TODO: Add your driver code here
    int i;
    //定义指向UART发送FIFO和状态寄存器的指针
    volatile unsigned int *uart_tx_fifo = uart + UART_TX_FIFO/sizeof(unsigned int);
    volatile unsigned int *uart_status = uart + UART_STATUS/sizeof(unsigned int);

    //循环遍历字符串中的每个字符，直到遇到空字符'\0'
    for (i = 0; s[i] != '\0'; i++) {
        /* if tx_fifo is full, loop (如果发送FIFO已满，则循环等待) */
        while (*uart_status & UART_TX_FIFO_FULL)
            ; //空循环体，持续检查
        *uart_tx_fifo = (unsigned int)s[i]; //将字符写入发送FIFO
    }

    return i; //返回发送的字符数
}

```

图 11: printf.c

三、 学习总结(总结设计、开发与调试过程中遇到的问题,及实验中的收获与不足)

1 问题总结(实验过程中遇到的问题、对问题的思考过程及解决方法)

比如 RTL 代码中出现的逻辑 bug、逻辑仿真和 FPGA 调试过程中的难点等。

出错模块指设计章节中列出的各个模块。

问题类别包括逻辑电路设计、RTL 语法、行为仿真和逻辑综合。

产生原因包括 xxx 概念理解不清、xxx 模块设计环节出现 xxx 问题、xxx 设计不符合 xxx 设计规范等。

● MIPS/RISC-V 定制功能型处理器问题补充。

排查问题 1 时,考虑到了该问题可能是之前向内存写入的数据出错,之后读出了错误数据导致的。

为此修改了 lwl,lwr 在边界条件上的字节偏移逻辑,但是所有情况下该修改都未对波形产生影响。

部分原有设计不符合 Verilator 语法,均以修改。

序号	问题类别	出错模块	问题现象	产生原因
1	逻辑电路设计	ALU	寄存器读出数据(向内存写入的数据)在某个周期后大量出错	ALU 或非逻辑
2	逻辑电路设计	状态机	握手信号拉高过早	jr 指令未进入写回状态
3	逻辑电路设计	状态机	CPU 运行过慢	NOP 指令未处理,会完整周期
4	逻辑电路设计	状态机	Inst_Ready 信号拉高时机不对	该信号在 INIT 状态

其中问题 1,问题 2 是由 FPGA 加速仿真找出的。



图 12: 问题 1 提到的写入数据出错,该图中的大量 trace_mismatch 均为该问题引起

```
assign NOR = ~ A ^ B; //按位同或, 101
```

图 13: 具体出错代码

表 1: 实验问题总结

2 收获总结(在数字电路设计方法、RTL 编写方法、数字电路验证方法及其他方面的收获)

本次调试过程中,主要问题出在 ALU 的或非运算,但是该问题仅在 microbench 中的 queen 测试例出现,在其余 39 个测试例中均未出现。推测原因可能是 NOR 指令仅在 queen 测试例中用到,或本次实验的仿真测试不在寄存器堆写入时做检查报错,仅检查内存读写相关的数据和信号,而 queen 测试例恰好向内存写入了相关数据。

类别	名称	体会	后续计划
代码编写	基础部件	某些的基础部件出错难以在调试时发现	编写代码时必须细心
代码调试	调试方法	对测试集本身的了解不够	了解部分测试集的特点,常用指令

表 2: 收获总结

四、 扩展思考(对讲义中思考题(如有)的理解和回答)

- 思考题 1:**处理器内部是否需要实现异步串行通信协议？

不需要；因为处理器内部通信要求低延迟和高带宽，而处理器内部部件物理距离极短，直接使用并行总线更高效。
- 思考题 2:**处理器如何访问 UART 控制器的 I/O 端口？

系统将 UART 控制器的寄存器分配到处理器内存地址空间中的特定一段地址，处理器直接使用通用的访存指令即可访问这些地址。

当处理器发出访问请求时,地址译码器识别出当前地址属于 UART 的映射区域,随后产生片选信号,将数据总线的读写请求引导至 UART 控制器而非物理内存。

五、 实验时长

在课后,你花费了大约 6 小时完成此次实验,其中各个阶段的用时如表 3所示。

步骤	时长(小时)	说明
设计	0.5	重新构建状态机,引入握手信号
开发	1.5	软件和硬件代码编写
调试	4	使用了加速仿真

表 3: 实验时长